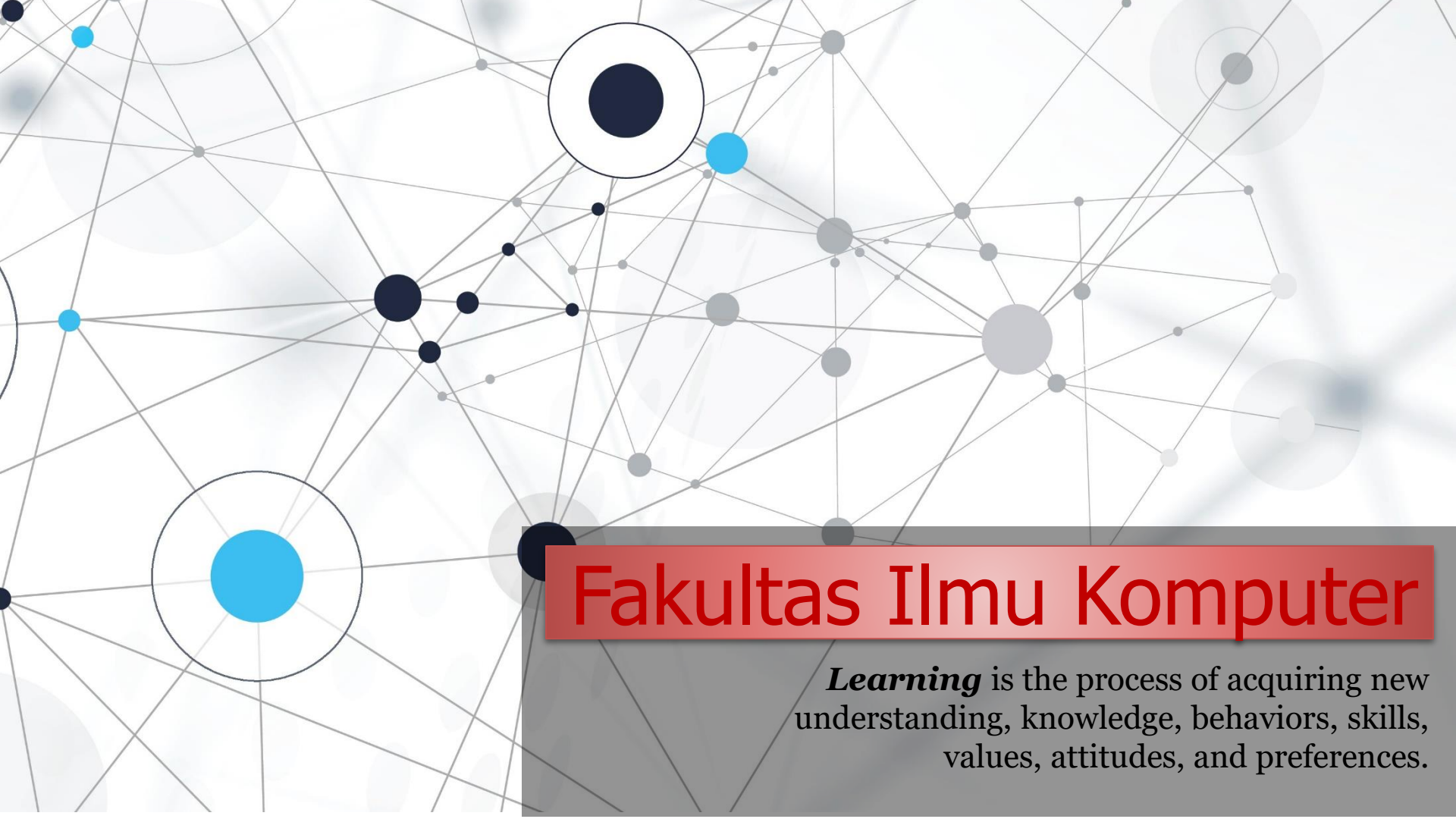


A complex network diagram with numerous nodes of varying sizes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circular patterns.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Hide Methods from Public Access

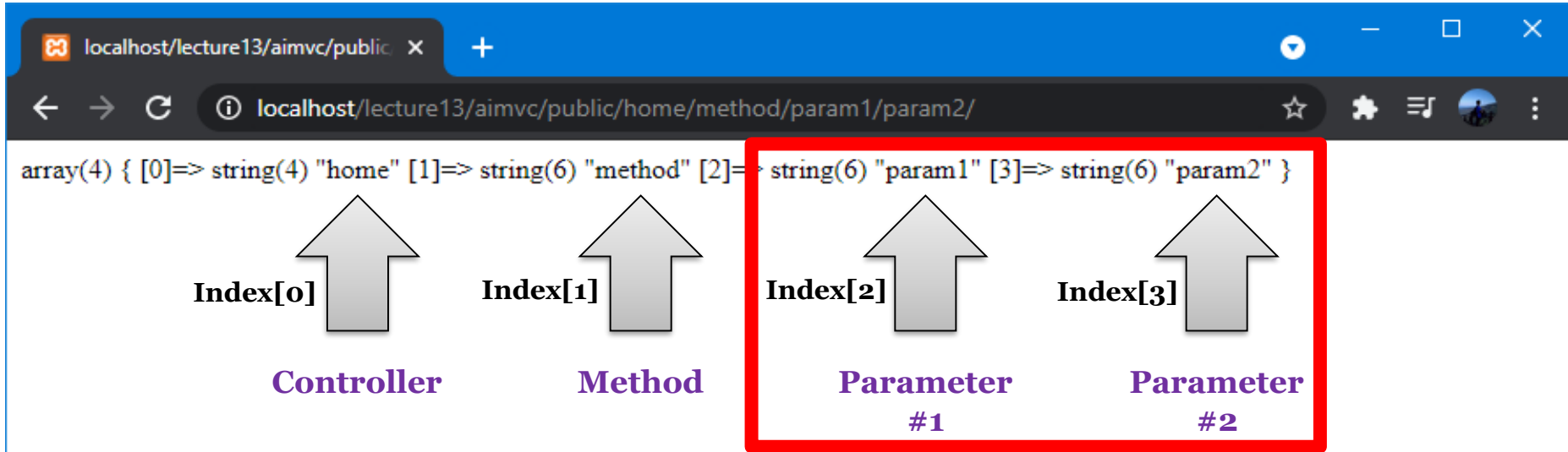


A decorative border of green leaves and branches at the top of the slide.

1 - Pass the value via URL

A decorative border of green leaves and branches at the bottom of the slide.

Handle Input Data from URL



Our app starts from this directory location:
<http://localhost/lecture13/aimvc/public/>

This means Controller default (home) and method default (index).

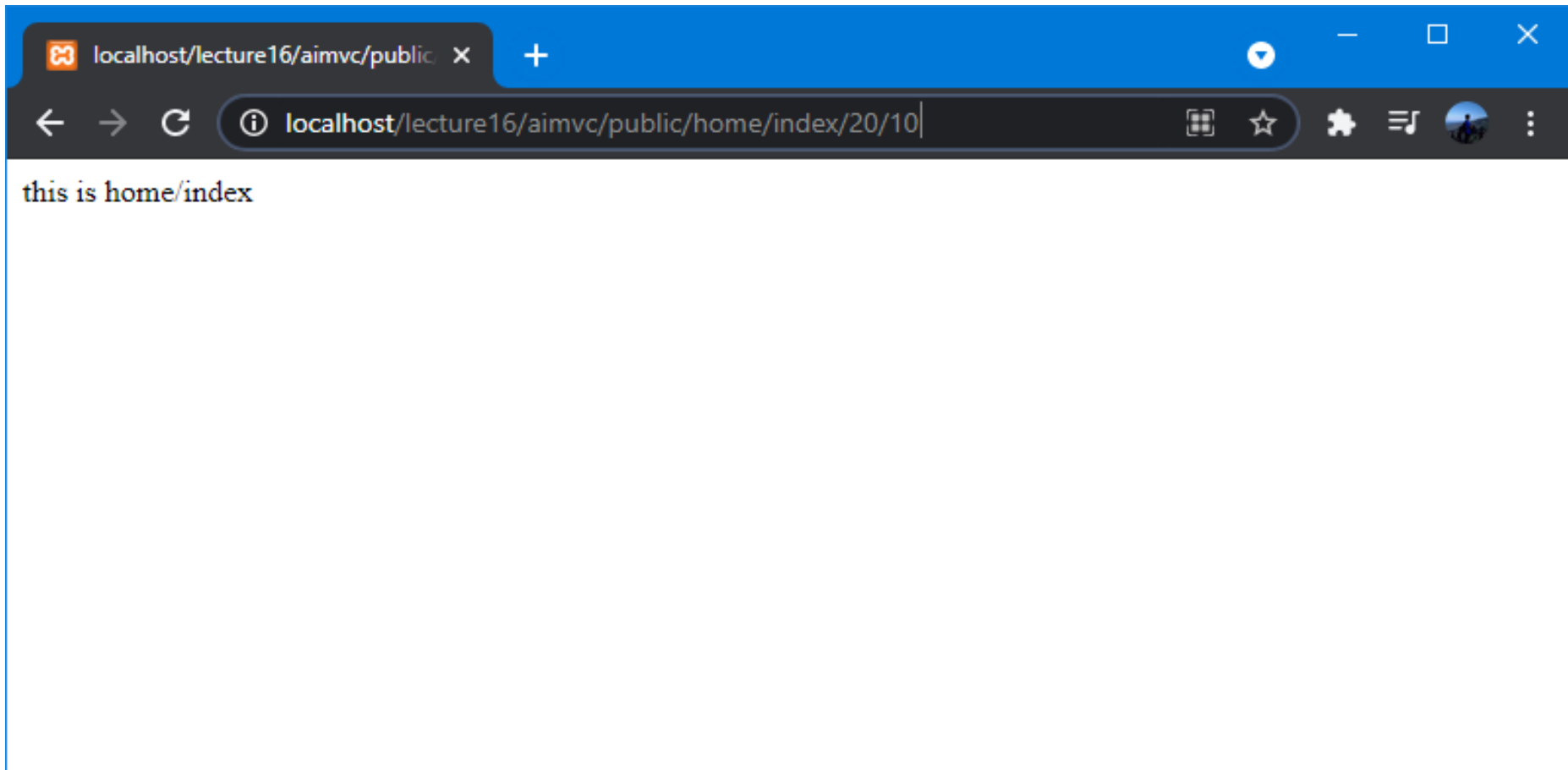
URL Format: <http://localhost/lecture13/aimvc/public/class/method/param1/param2/>

Handle Input Data from Default Method

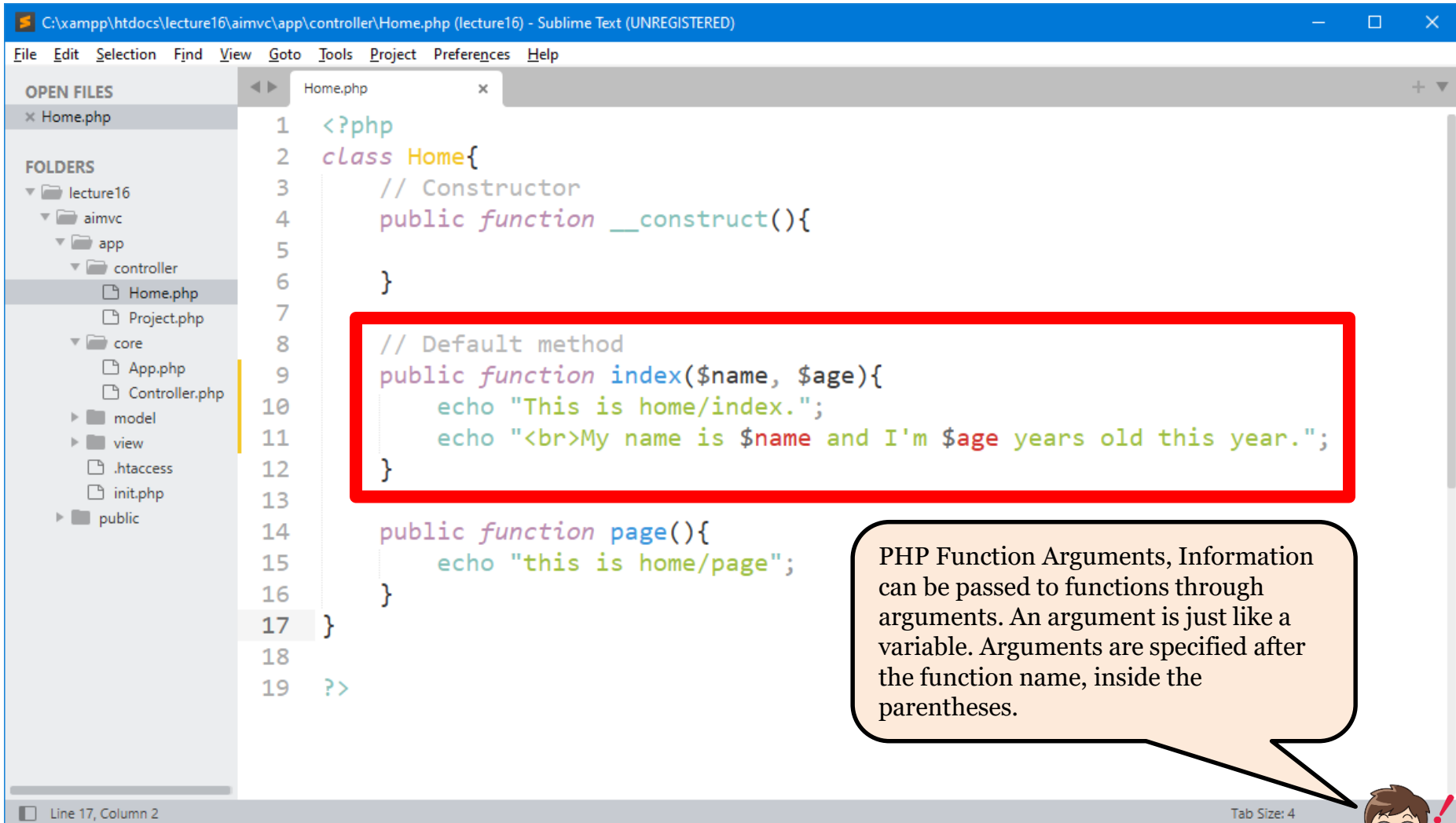
Default method can be accessed through these URLs:

<http://localhost/lecture16/aimvc/public/home/index/>

<http://localhost/lecture16/aimvc/public/home/>



Handle Input Data from Default Method



C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php (lecture16) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

- Home.php

FOLDERS

- lecture16
 - aimvc
 - app
 - controller
 - Home.php
 - Project.php
 - core
 - App.php
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public

```
1 <?php
2 class Home{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index($name, $age){
10         echo "This is home/index.";
11         echo "<br>My name is $name and I'm $age years old this year.";
12     }
13
14     public function page(){
15         echo "this is home/page";
16     }
17 }
18
19 ?>
```

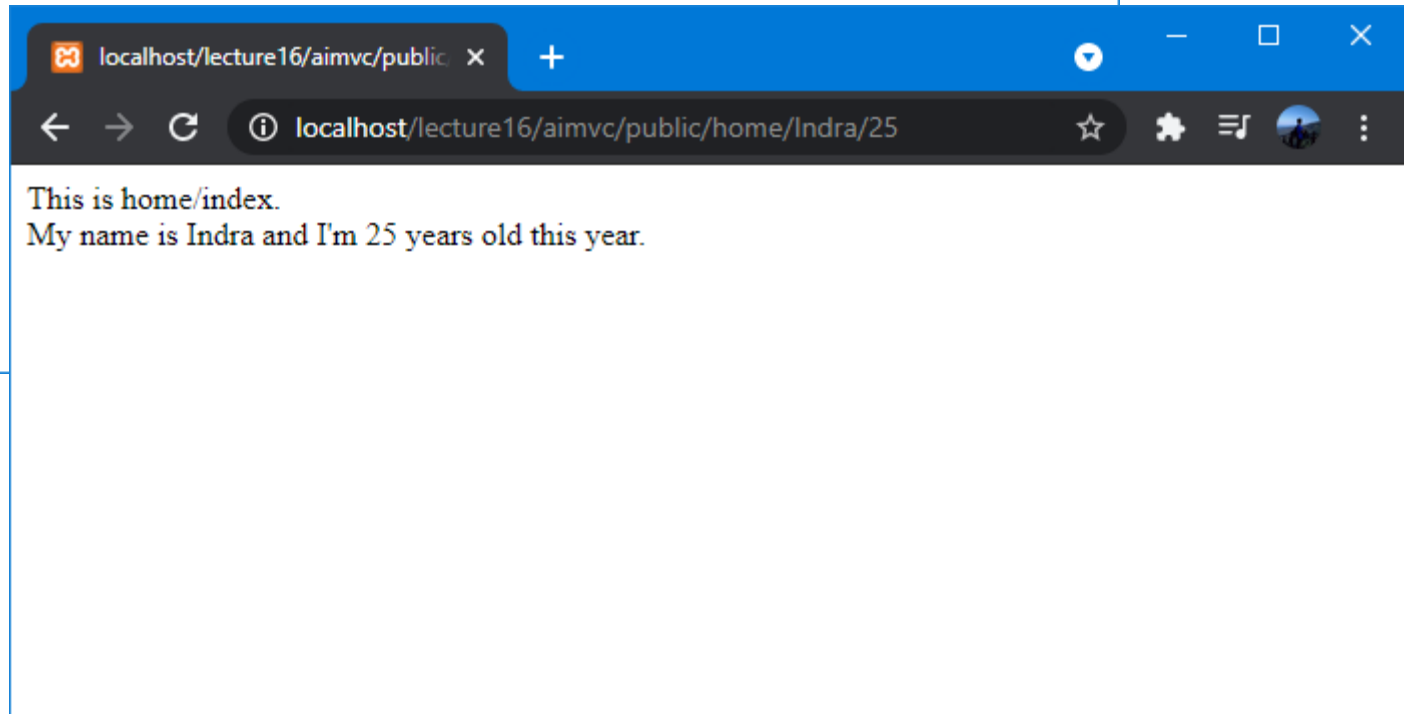
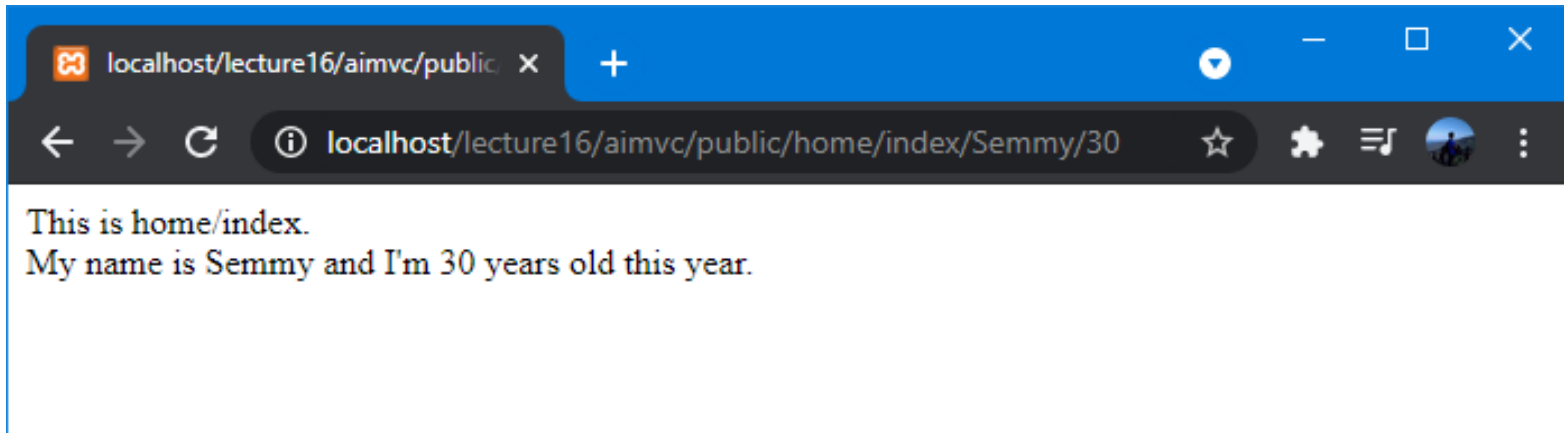
PHP Function Arguments, Information can be passed to functions through arguments. An argument is just like a variable. Arguments are specified after the function name, inside the parentheses.

Line 17, Column 2

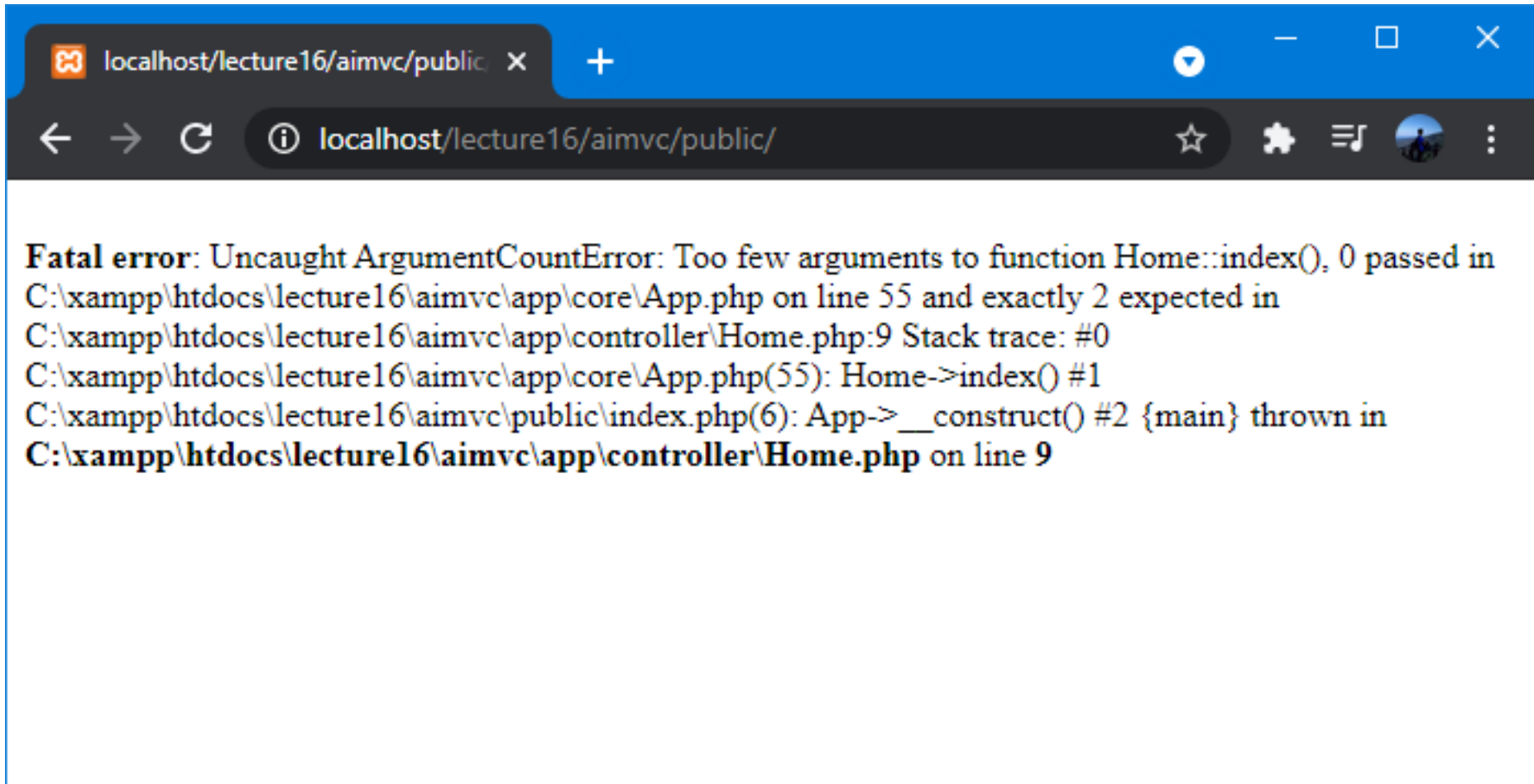
Tab Size: 4



Handle Input Data from Default Method



Why Fatal Error?

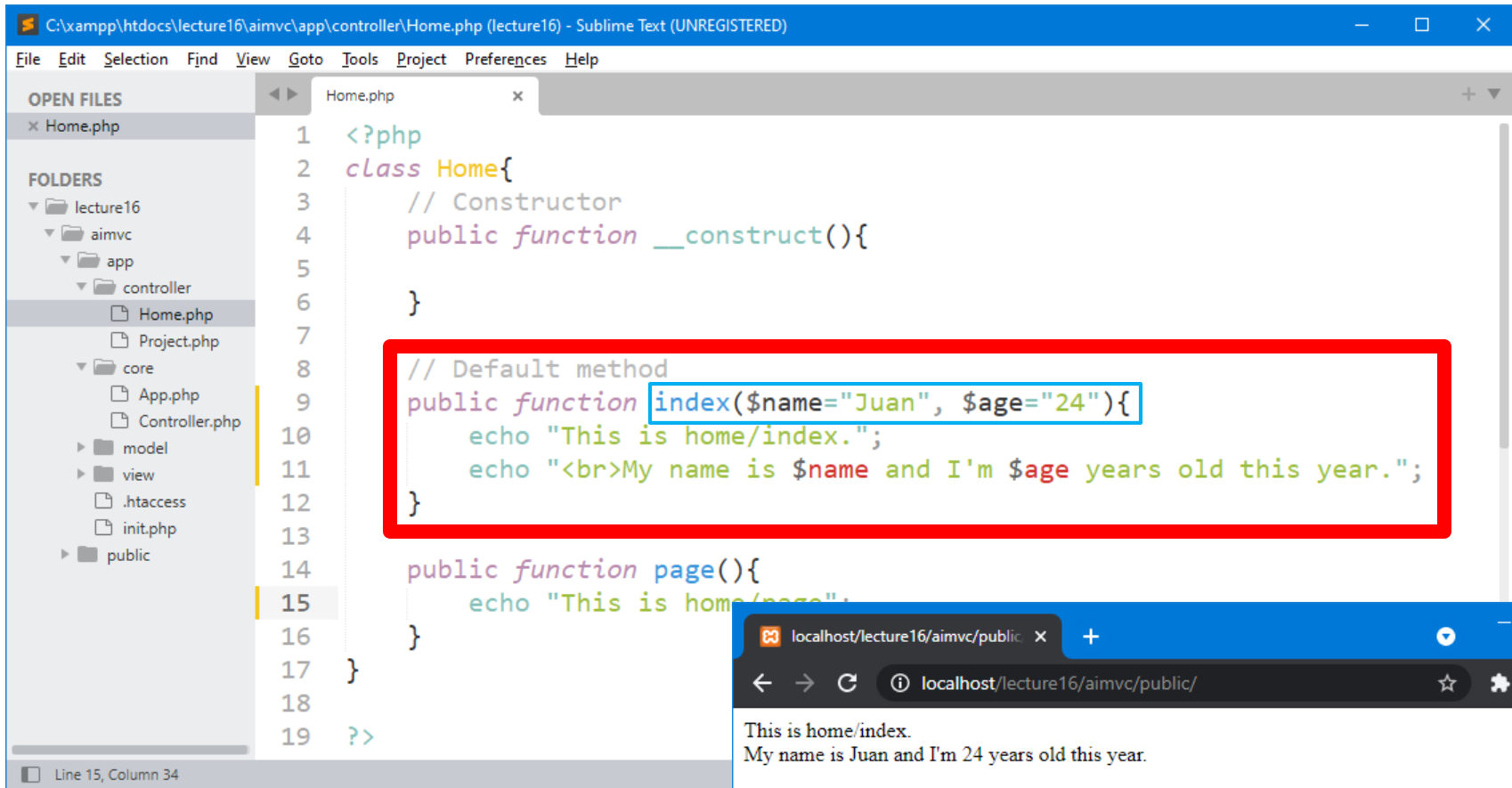


The screenshot shows a web browser window with a single tab titled 'localhost/lecture16/aimvc/public'. The address bar shows the URL 'localhost/lecture16/aimvc/public/'. The main content area displays a fatal error message in black text on a white background. The error message reads: 'Fatal error: Uncaught ArgumentCountError: Too few arguments to function Home::index(), 0 passed in C:\xampp\htdocs\lecture16\aimvc\app\core\App.php on line 55 and exactly 2 expected in C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php:9 Stack trace: #0 C:\xampp\htdocs\lecture16\aimvc\app\core\App.php(55): Home->index() #1 C:\xampp\htdocs\lecture16\aimvc\public\index.php(6): App->__construct() #2 {main} thrown in C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php on line 9'. The browser's interface includes a blue header bar with window controls, a dark address bar, and a toolbar with navigation and utility icons.

```
Fatal error: Uncaught ArgumentCountError: Too few arguments to function Home::index(), 0 passed in  
C:\xampp\htdocs\lecture16\aimvc\app\core\App.php on line 55 and exactly 2 expected in  
C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php:9 Stack trace: #0  
C:\xampp\htdocs\lecture16\aimvc\app\core\App.php(55): Home->index() #1  
C:\xampp\htdocs\lecture16\aimvc\public\index.php(6): App->__construct() #2 {main} thrown in  
C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php on line 9
```

There're no default parameters in this method. You may need to remember this, when you create a method with parameters, you should provide the method with default parameters.

Default Values for Function Parameters



The image shows a code editor window titled "C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php (lecture16) - Sublime Text (UNREGISTERED)". The editor displays the following PHP code:

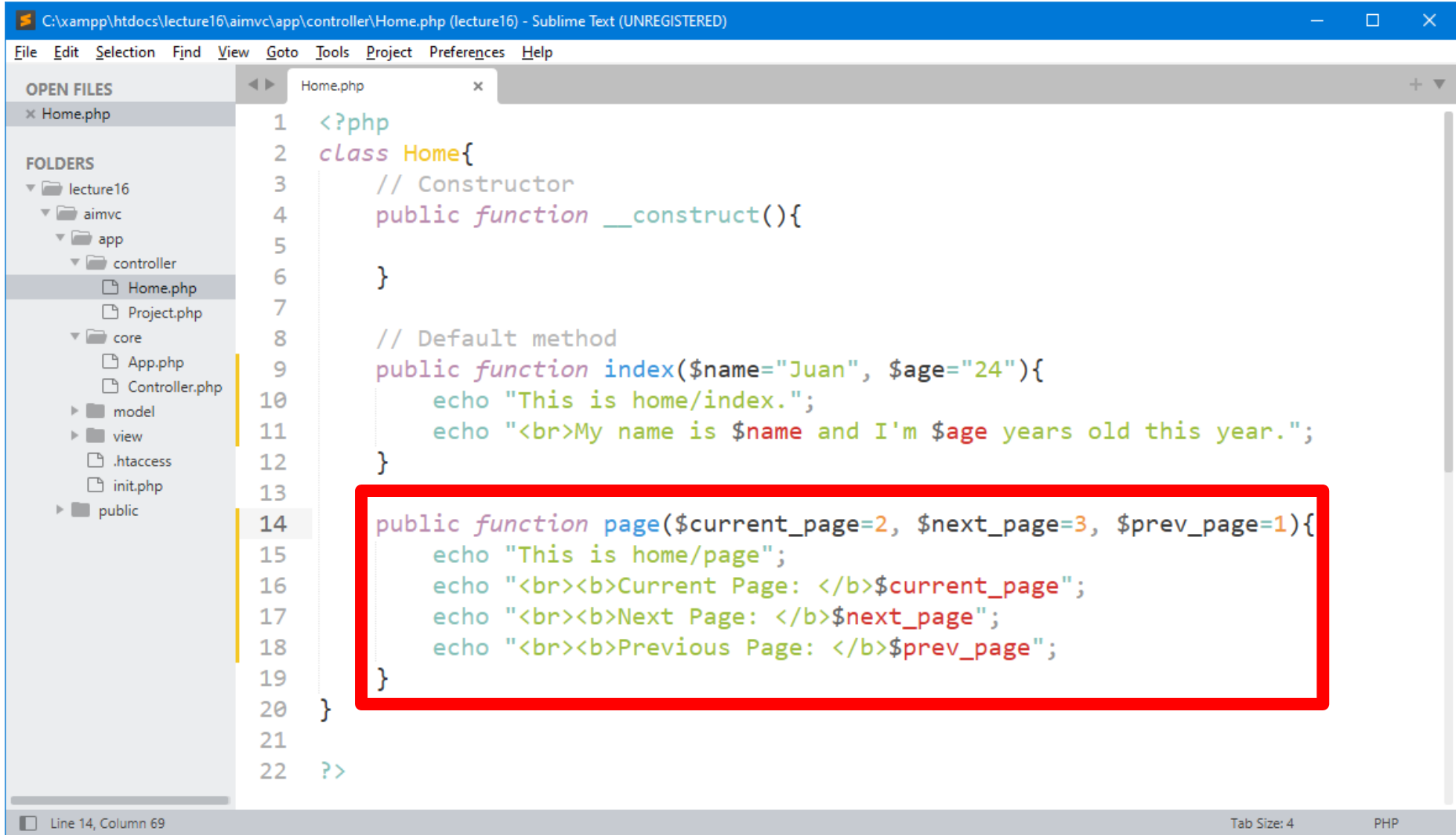
```
1 <?php
2 class Home{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index($name="Juan", $age="24"){
10         echo "This is home/index.";
11         echo "<br>My name is $name and I'm $age years old this year.";
12     }
13
14     public function page(){
15         echo "This is home/page.";
16     }
17 }
18
19 ?>
```

The code is highlighted with a red box around the `index` method. The browser window shows the output of the `index` method:

localhost/lecture16/aimvc/public/

This is home/index.
My name is Juan and I'm 24 years old this year.

Many Arguments from URL Request



The screenshot shows a Sublime Text editor window titled "C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php (lecture16) - Sublime Text (UNREGISTERED)". The editor displays a PHP class named `Home` with the following code:

```
1 <?php
2 class Home{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index($name="Juan", $age="24"){
10         echo "This is home/index.";
11         echo "<br>My name is $name and I'm $age years old this year.";
12     }
13
14     public function page($current_page=2, $next_page=3, $prev_page=1){
15         echo "This is home/page";
16         echo "<br><b>Current Page: </b>$current_page";
17         echo "<br><b>Next Page: </b>$next_page";
18         echo "<br><b>Previous Page: </b>$prev_page";
19     }
20 }
21
22 ?>
```

The `page()` method (lines 14-19) is highlighted with a red box, illustrating how multiple arguments can be passed to a function from a URL request.

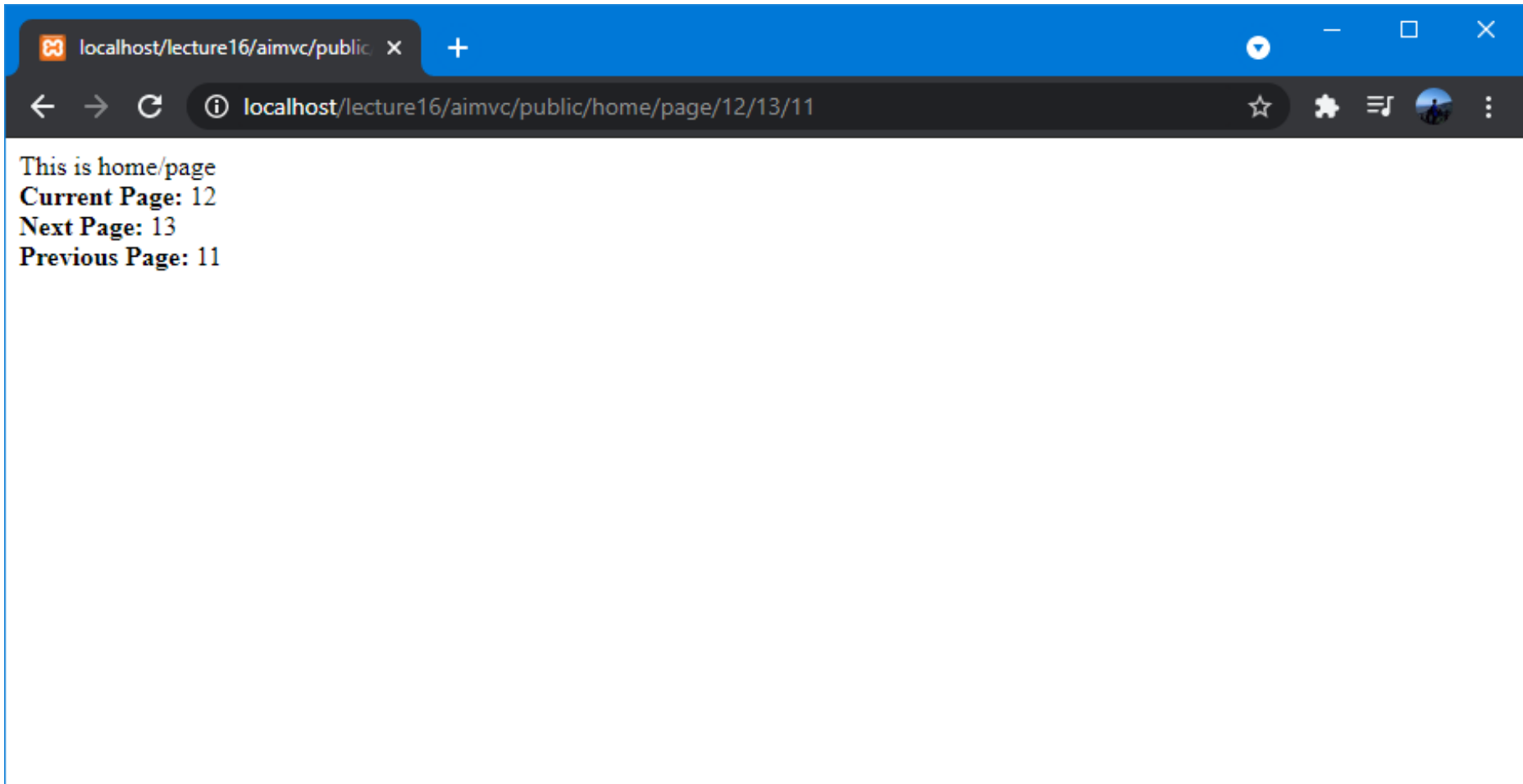
Line 14, Column 69

Tab Size: 4

PHP

You can add as many arguments as you want, just separate them with a comma.

Many Arguments from URL Request

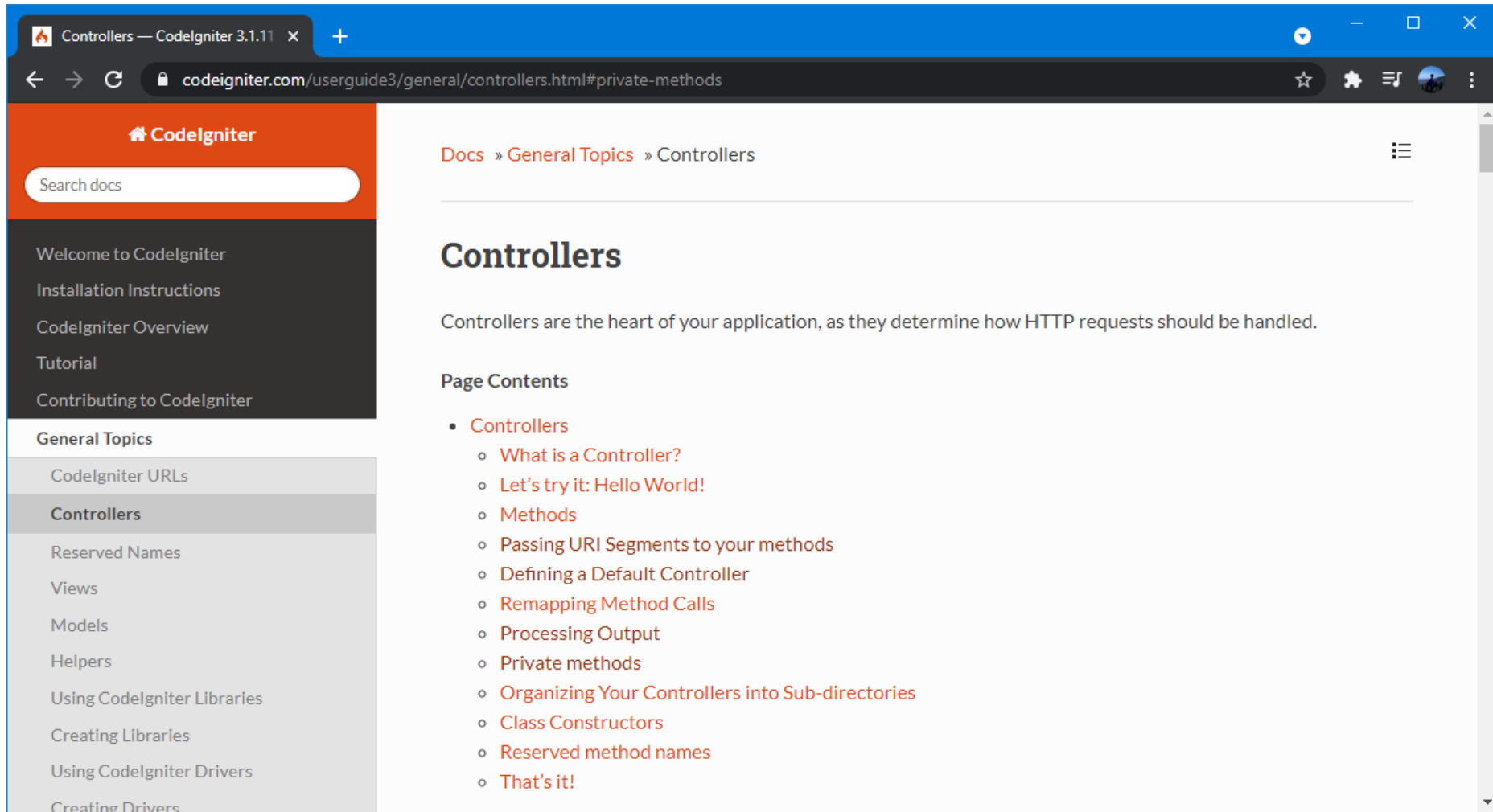


We added three arguments as a parameter input for the method in class controller, we separate data with a comma.

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green with some showing signs of aging or damage. A large, bold, black serif font displays the text "2 - Prevent Direct Access Methods" in the center.

2 - Prevent Direct Access Methods

Controllers in CodeIgniter



The screenshot shows a web browser displaying the CodeIgniter 3.1.11 documentation page for Controllers. The browser's address bar shows the URL `codeigniter.com/userguide3/general/controllers.html#private-methods`. The page has a blue header with the CodeIgniter logo and a search bar. The left sidebar contains a navigation menu with links to 'Welcome to CodeIgniter', 'Installation Instructions', 'CodeIgniter Overview', 'Tutorial', and 'Contributing to CodeIgniter'. Below this, there is a 'General Topics' section with a list of links: 'CodeIgniter URLs', 'Controllers' (which is highlighted), 'Reserved Names', 'Views', 'Models', 'Helpers', 'Using CodeIgniter Libraries', 'Creating Libraries', 'Using CodeIgniter Drivers', and 'Creating Drivers'. The main content area has a breadcrumb trail 'Docs » General Topics » Controllers' and a title 'Controllers'. Below the title, a paragraph states: 'Controllers are the heart of your application, as they determine how HTTP requests should be handled.' This is followed by a 'Page Contents' section with a bulleted list of topics: 'Controllers', 'What is a Controller?', 'Let's try it: Hello World!', 'Methods', 'Passing URI Segments to your methods', 'Defining a Default Controller', 'Remapping Method Calls', 'Processing Output', 'Private methods', 'Organizing Your Controllers into Sub-directories', 'Class Constructors', 'Reserved method names', and 'That's it!'.

Controllers — CodeIgniter 3.1.11

codeigniter.com/userguide3/general/controllers.html#private-methods

CodeIgniter

Search docs

Welcome to CodeIgniter

Installation Instructions

CodeIgniter Overview

Tutorial

Contributing to CodeIgniter

General Topics

- CodeIgniter URLs
- Controllers**
- Reserved Names
- Views
- Models
- Helpers
- Using CodeIgniter Libraries
- Creating Libraries
- Using CodeIgniter Drivers
- Creating Drivers

Docs » General Topics » Controllers

Controllers

Controllers are the heart of your application, as they determine how HTTP requests should be handled.

Page Contents

- Controllers
 - What is a Controller?
 - Let's try it: Hello World!
 - Methods
 - Passing URI Segments to your methods
 - Defining a Default Controller
 - Remapping Method Calls
 - Processing Output
 - Private methods
 - Organizing Your Controllers into Sub-directories
 - Class Constructors
 - Reserved method names
 - That's it!

Link: <https://codeigniter.com/userguide3/general/controllers.html>

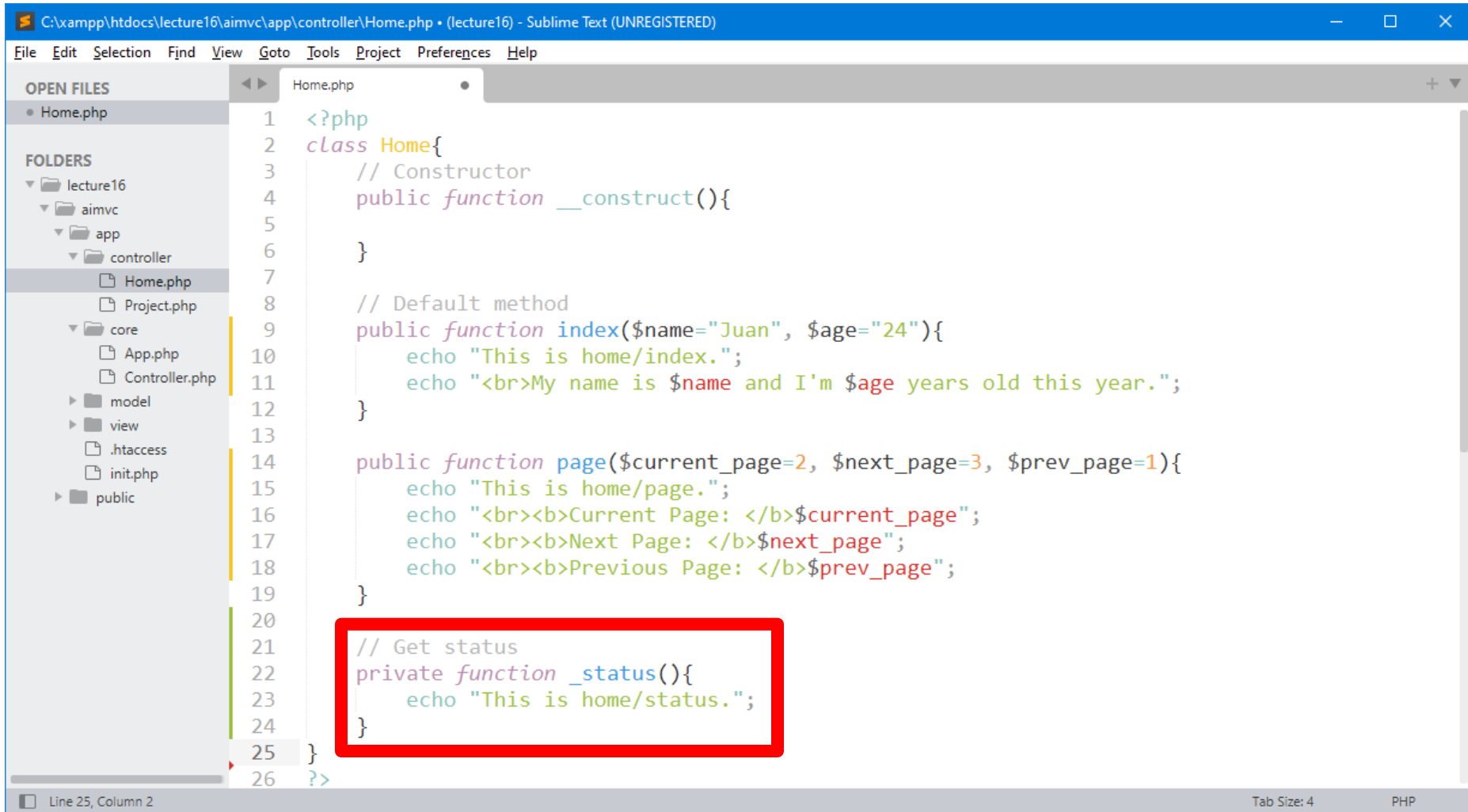
**How to prevent direct
access methods in
controller class?**

How to Prevent Direct Access Methods?

In some cases we may want certain methods hidden from public access, so there's no direct access to a method. In order to achieve this, simply declare the method as being ***private*** or ***protected*** and it will not be served via a URL request.

We can create *private* or *protected* functions in the class controller by using the ***underscore prefix***.

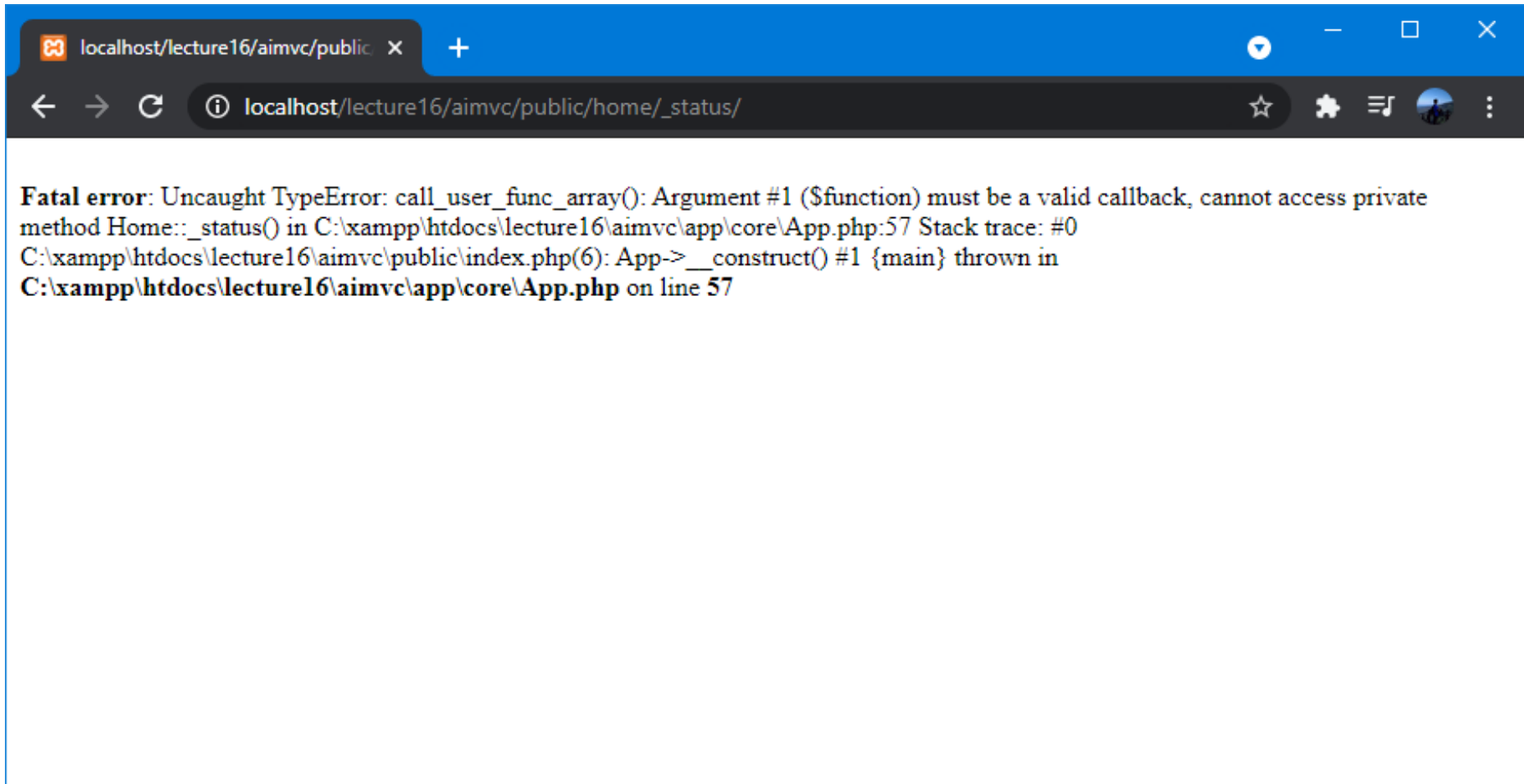
Close Direct Access - *Private Methods*



```
1 <?php
2 class Home{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index($name="Juan", $age="24"){
10         echo "This is home/index.";
11         echo "<br>My name is $name and I'm $age years old this year.";
12     }
13
14     public function page($current_page=2, $next_page=3, $prev_page=1){
15         echo "This is home/page.";
16         echo "<br><b>Current Page: </b>$current_page";
17         echo "<br><b>Next Page: </b>$next_page";
18         echo "<br><b>Previous Page: </b>$prev_page";
19     }
20
21     // Get status
22     private function _status(){
23         echo "This is home/status.";
24     }
25 }
26 ?>
```

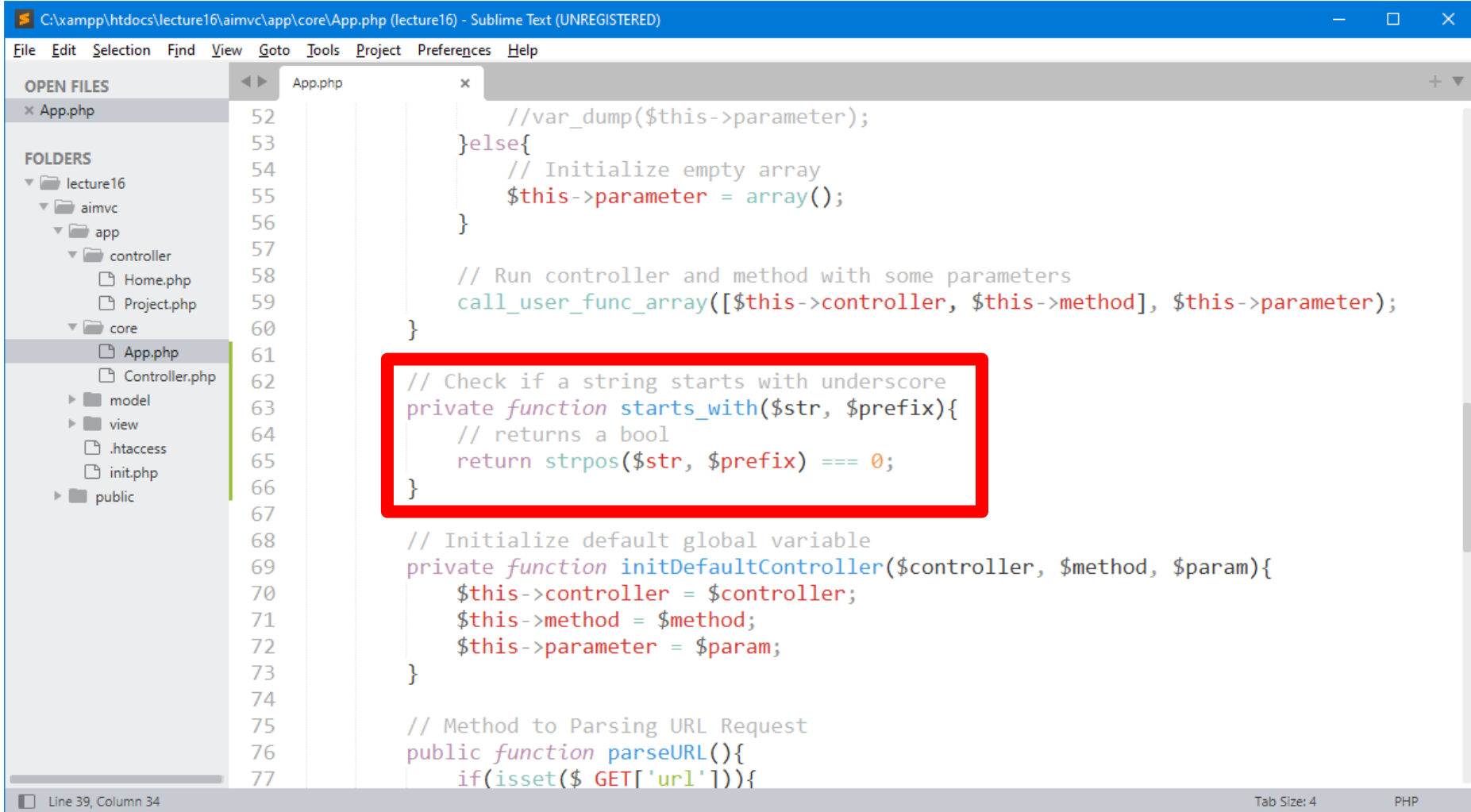
Prefixing method names with an ***underscore*** will also prevent them from being called from URL. This is a legacy feature that is left for backwards-compatibility.

Try to access private method via the URL



Browser returns an fatal error must be a valid callback, *cannot access private method*.

Create a Function to Check Underscore



```
C:\xampp\htdocs\lecture16\aimvc\app\core\App.php (lecture16) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

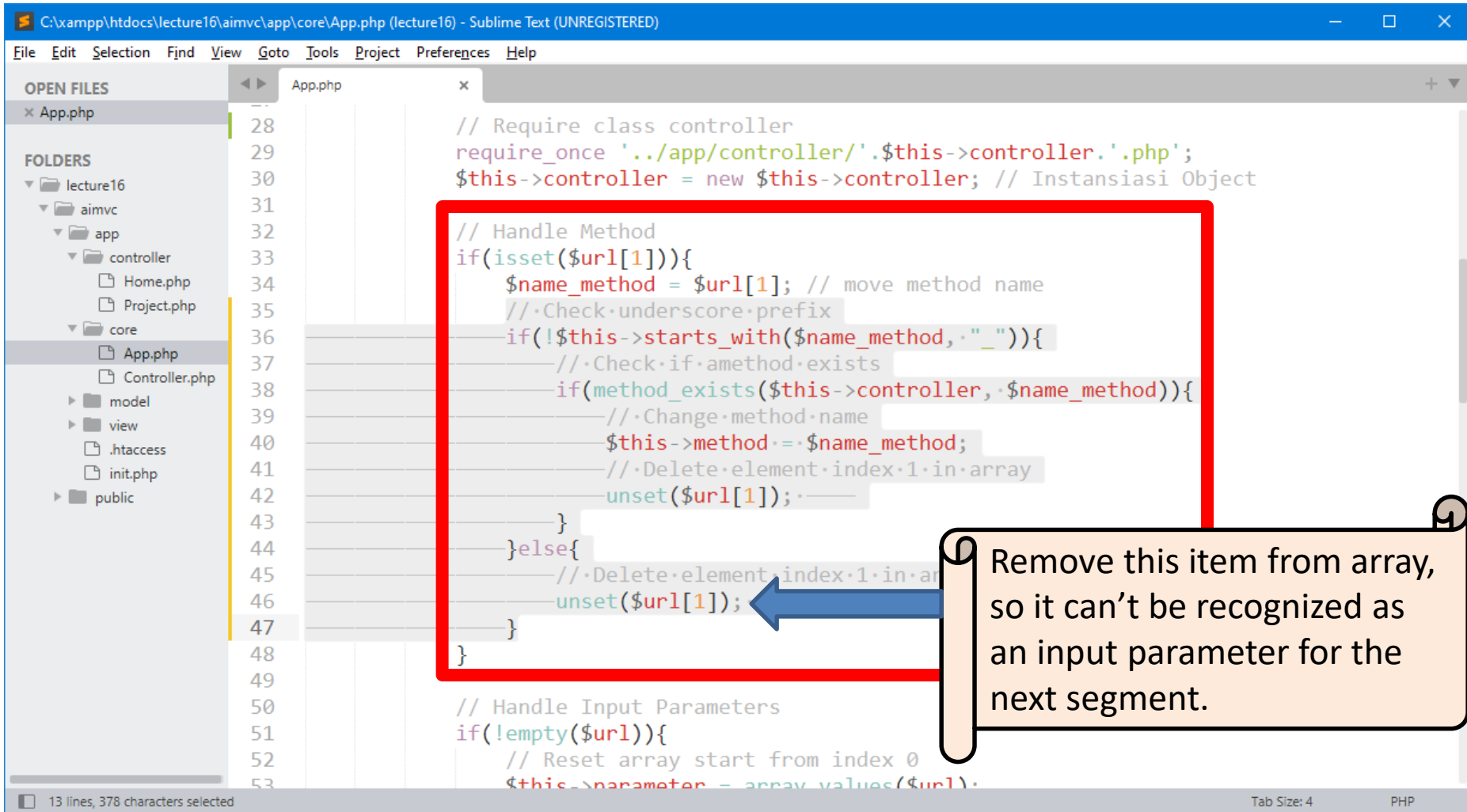
OPEN FILES
x App.php

FOLDERS
v lecture16
  v aimvc
    v app
      v controller
        Home.php
        Project.php
      v core
        App.php
        Controller.php
      v model
      v view
      .htaccess
      init.php
      public

52 //var_dump($this->parameter);
53 }else{
54 // Initialize empty array
55 $this->parameter = array();
56 }
57
58 // Run controller and method with some parameters
59 call_user_func_array([$this->controller, $this->method], $this->parameter);
60 }
61
62 // Check if a string starts with underscore
63 private function starts_with($str, $prefix){
64 // returns a bool
65 return strpos($str, $prefix) === 0;
66 }
67
68 // Initialize default global variable
69 private function initDefaultController($controller, $method, $param){
70 $this->controller = $controller;
71 $this->method = $method;
72 $this->parameter = $param;
73 }
74
75 // Method to Parsing URL Request
76 public function parseURL(){
77 if(isset($_GET['url'])){
```

Line 39, Column 34 Tab Size: 4 PHP

Update *your* Core App.php File



C:\xampp\htdocs\lecture16\aimvc\app\core\App.php (lecture16) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
x App.php

FOLDERS
lecture16
aimvc
app
controller
Home.php
Project.php
core
App.php
Controller.php
model
view
.htaccess
init.php
public

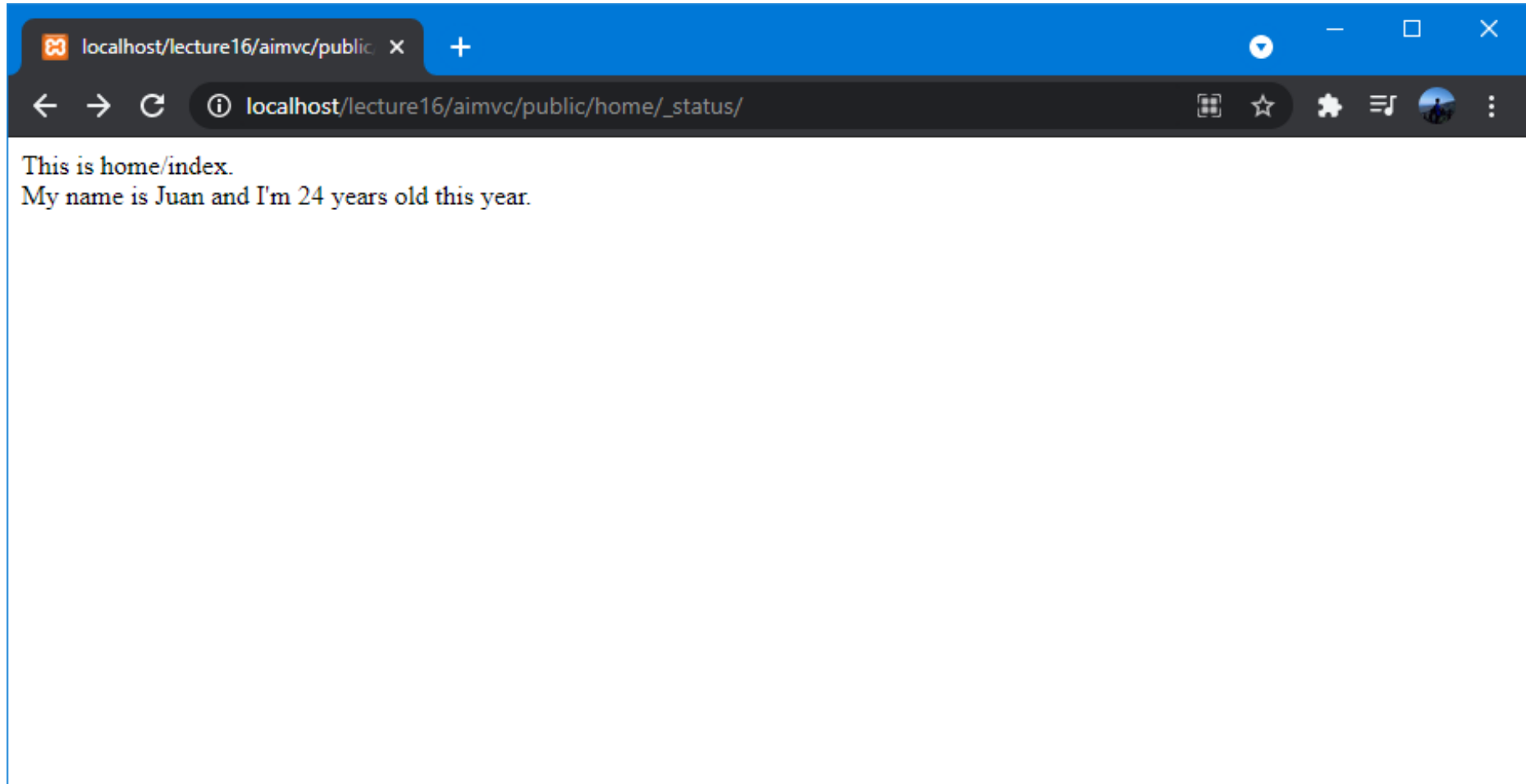
```
28 // Require class controller
29 require_once '../app/controller/'.$this->controller.'.php';
30 $this->controller = new $this->controller; // Instansiasi Object
31
32 // Handle Method
33 if(isset($url[1])){
34     $name_method = $url[1]; // move method name
35     //Check underscore prefix
36     if(!$this->starts_with($name_method, "_")){
37         //Check if a method exists
38         if(method_exists($this->controller, $name_method)){
39             //Change method name
40             $this->method = $name_method;
41             //Delete element index 1 in array
42             unset($url[1]);
43         }
44     }else{
45         //Delete element index 1 in array
46         unset($url[1]);
47     }
48 }
49
50 // Handle Input Parameters
51 if(!empty($url)){
52     // Reset array start from index 0
53     $this->parameter = array_values($url);
```

Remove this item from array, so it can't be recognized as an input parameter for the next segment.

13 lines, 378 characters selected

Tab Size: 4 PHP

Final Testing - Direct Access Methods



Now, we can completely Prevent Direct Access Methods from URL and even try to handle underscore prefix from core App.php file.



exercise for students

Latihan untuk *Students*

Create directory : C:\xampp\htdocs\lecture16\
Create file : __.php

Buat semua contoh yang sudah di pelajari sebelumnya.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

